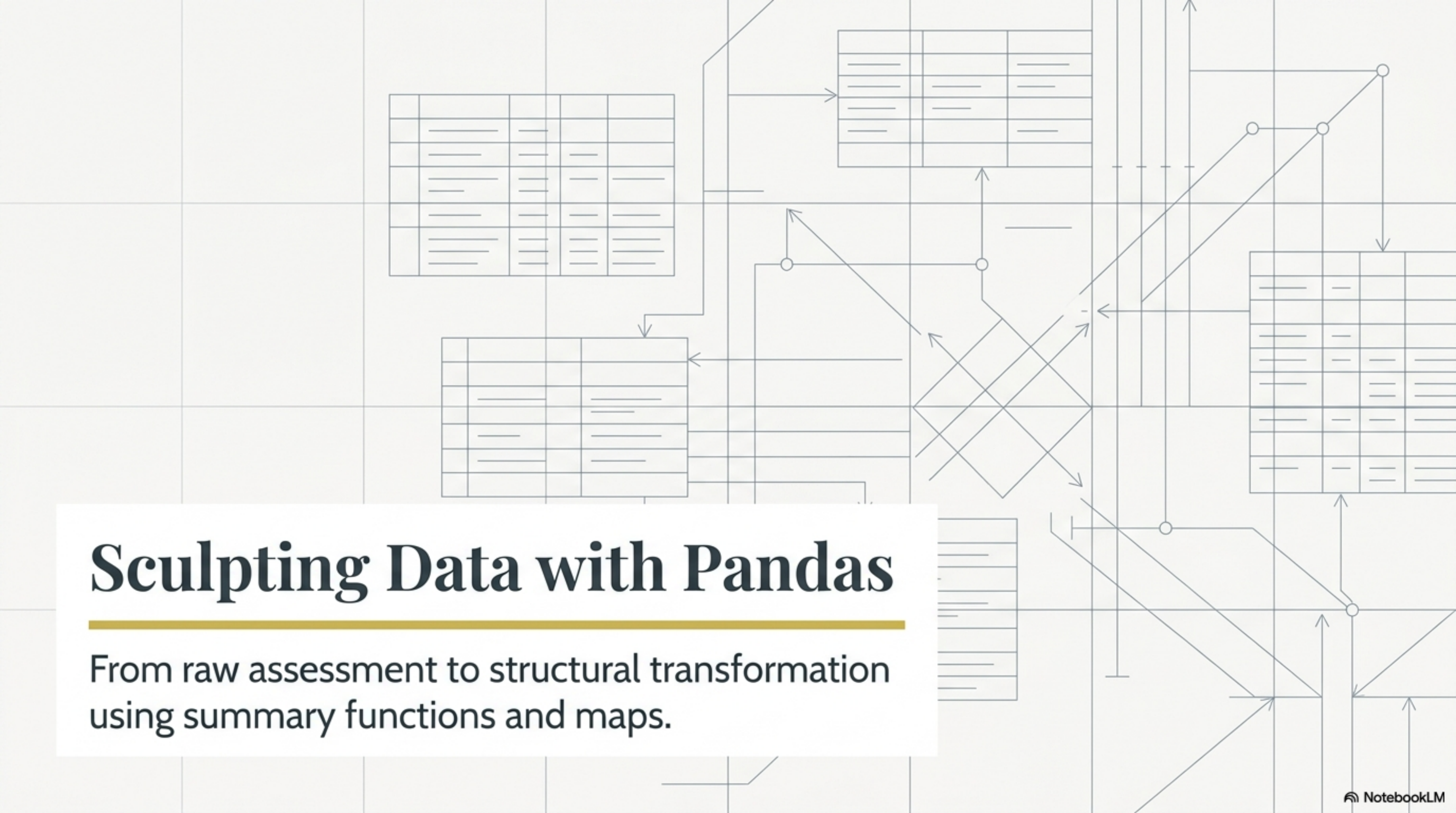


The background features a light gray grid. Overlaid on this grid are several faint, stylized tables and a network of thin gray lines with small circular nodes at the intersections. The tables vary in size and orientation, some appearing as if they are part of a larger data structure being manipulated. The lines and nodes suggest a flow or transformation process, consistent with the theme of data sculpting.

Sculpting Data with Pandas

From raw assessment to structural transformation using summary functions and maps.

The analytical workflow follows a medical intervention



Stage 1: Diagnose (Summary Functions)

We cannot manipulate what we do not understand. Extract baselines, distributions, and unique identities from raw memory.



Stage 2: Treat (Maps & Apply)

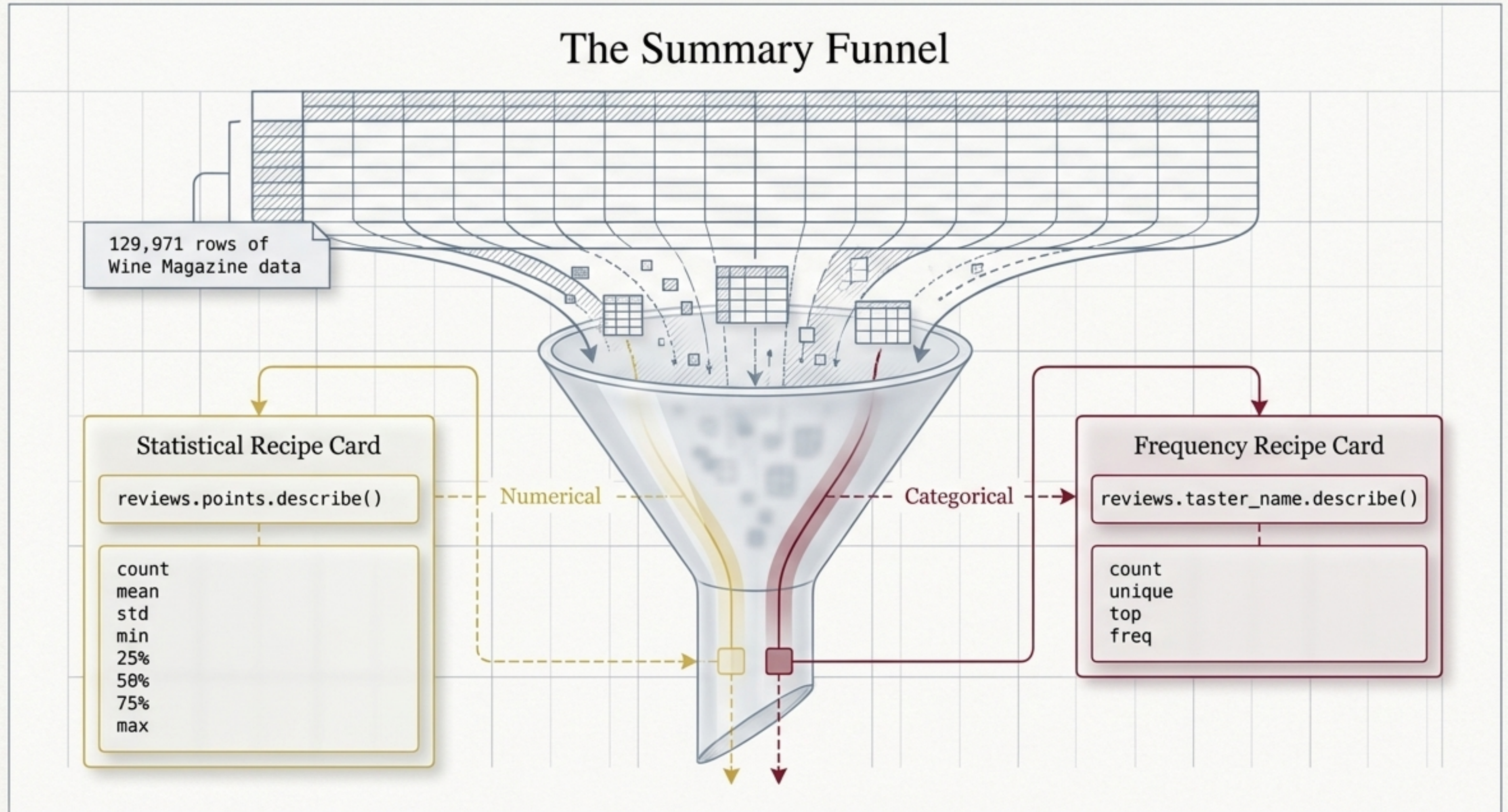
Construct mathematical or logical functions to reshape the raw input into analysis-ready formats.



Stage 3: Optimise (Built-ins)

Bypass standard functional loops to leverage pandas' high-speed internal architecture.

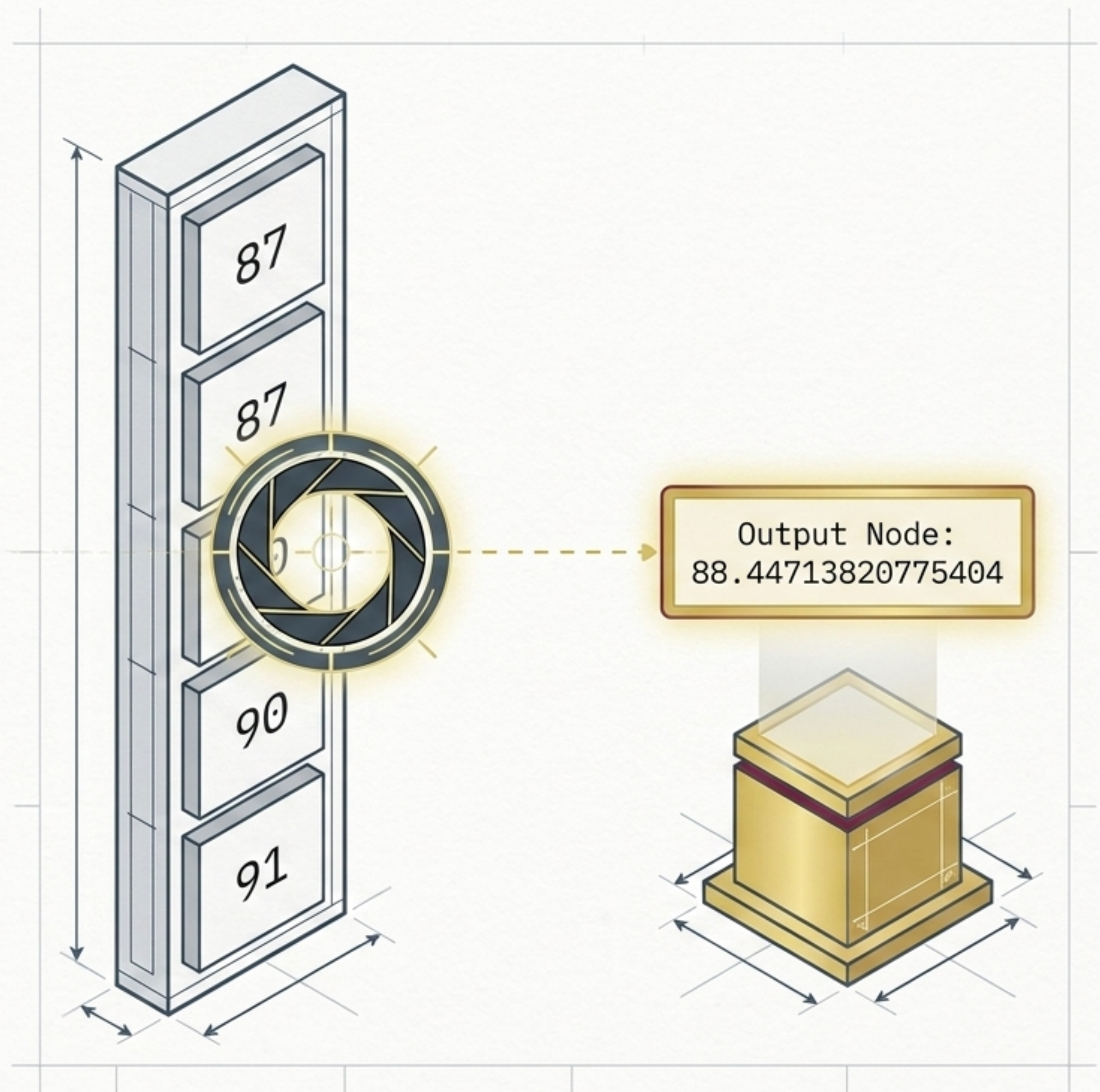
High-level profiling adapts to the data type



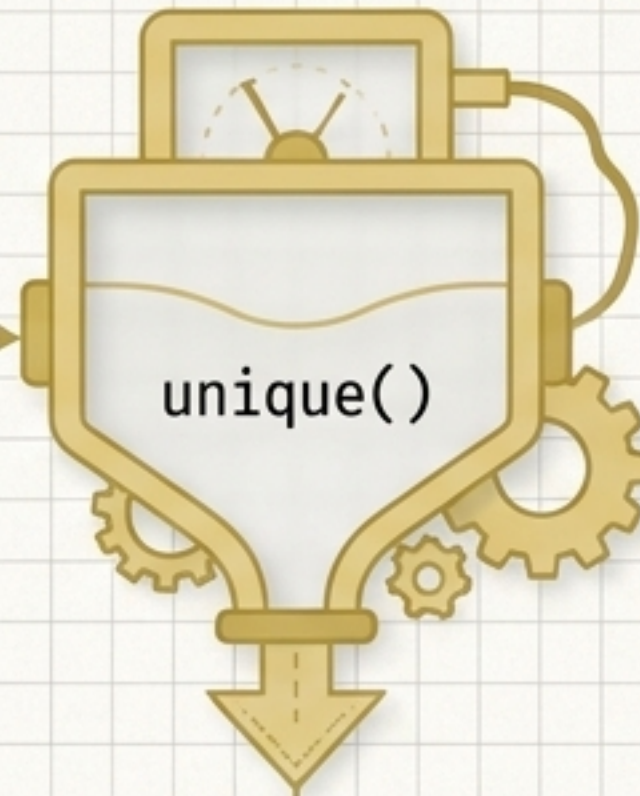
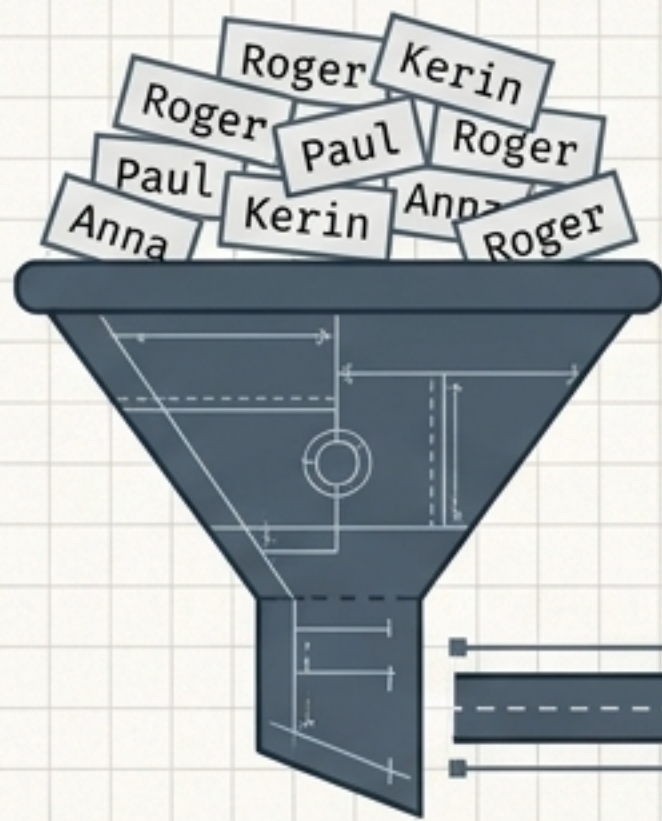
Extracting a pinpoint metric

When a full summary is too broad, pandas provides functions to extract single, actionable baseline metrics.

```
reviews.points.mean()
```



Constructing sets and measuring frequencies

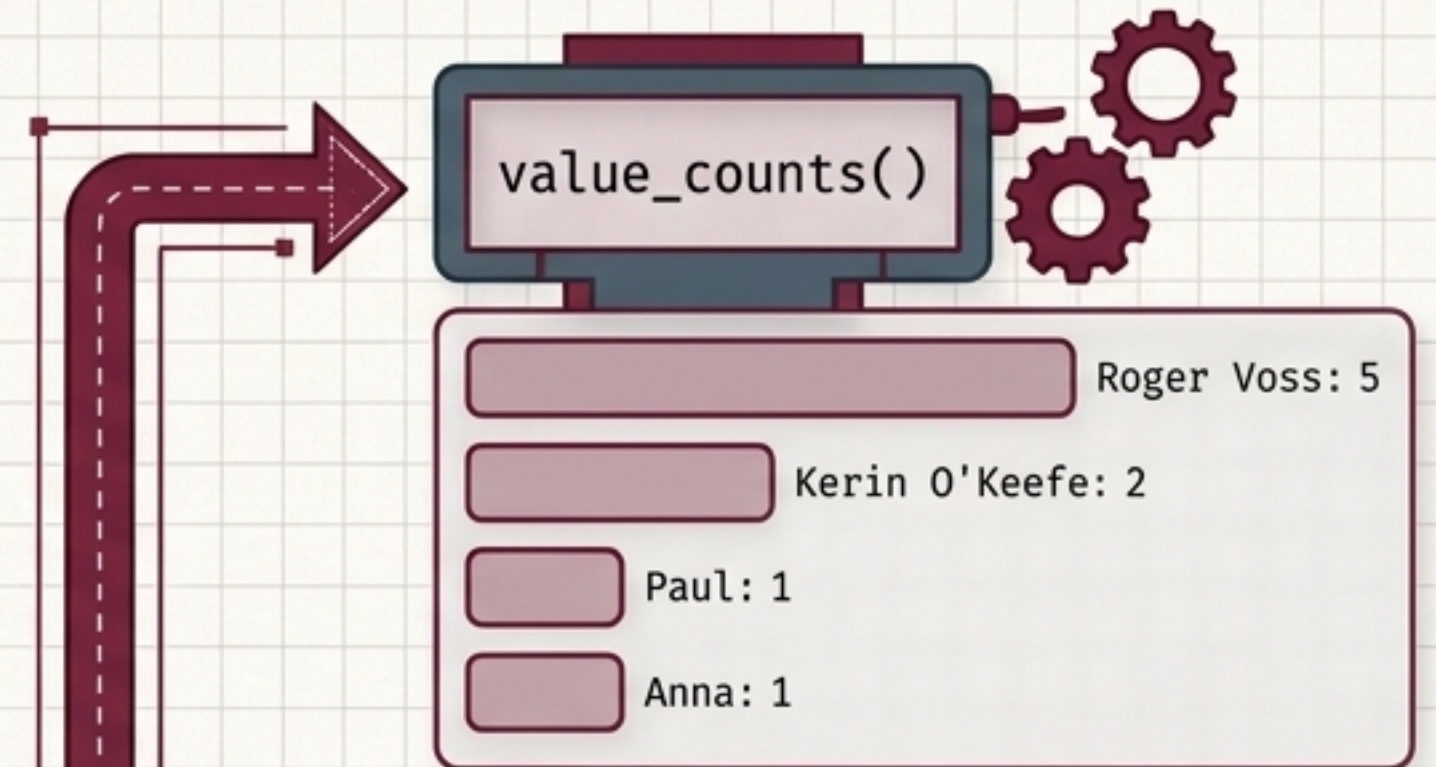


Kerin O'Keefe
Roger Voss
Paul
Anna

Operation 1: `unique()`

Returns an array of distinct values.

```
reviews.taster_name.unique()
```

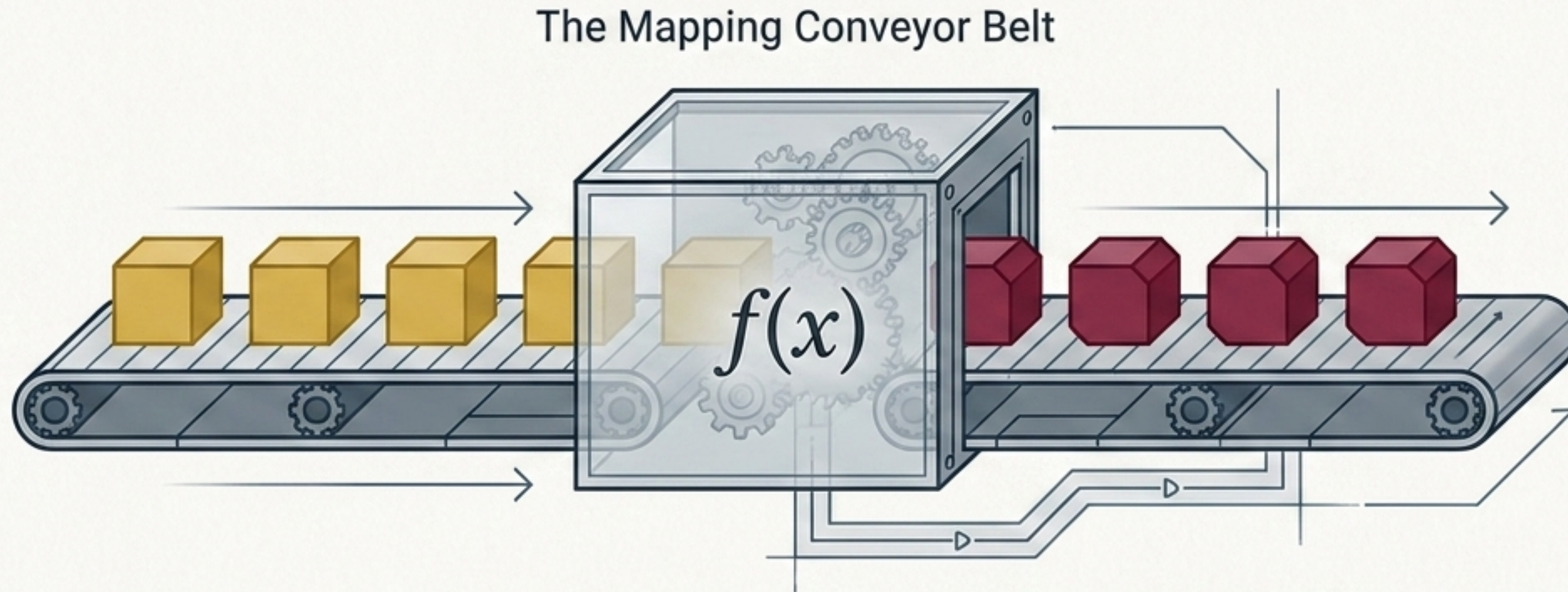


Operation 2: `value_counts()`

Returns distinct values and how often they occur, automatically sorted by frequency.

```
reviews.taster_name.value_counts()
```


A mathematical translation of values

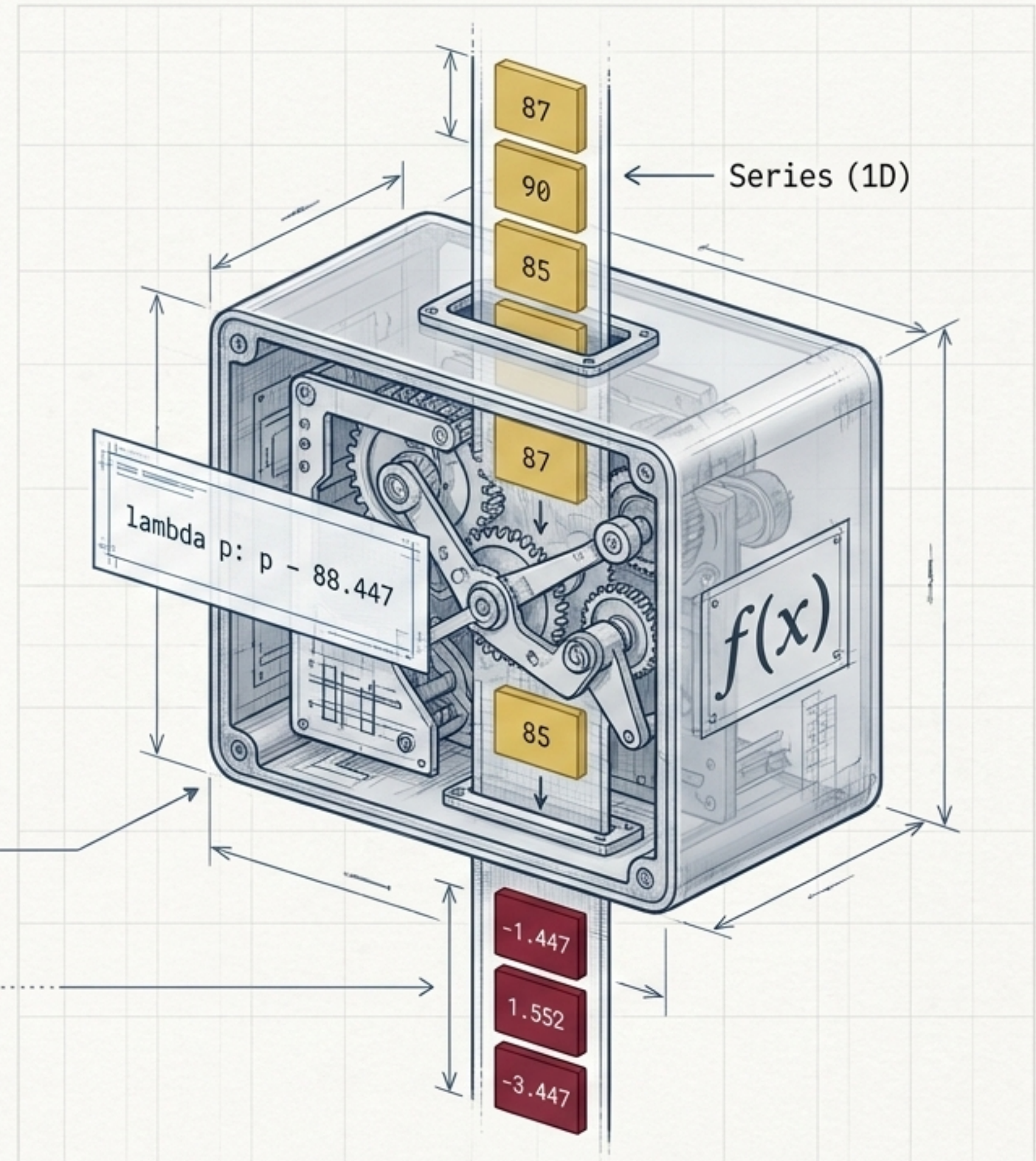


Borrowed from mathematics, a map takes one set of values and creates a new representation. In data science, this is the primary mechanism for reformatting raw inputs to match analytical requirements. The function expects a single value, transforms it, and returns it.

Precision edits on a single Series

`map()` is the simpler mapping method. It applies a custom transformation to every single value within a Series.

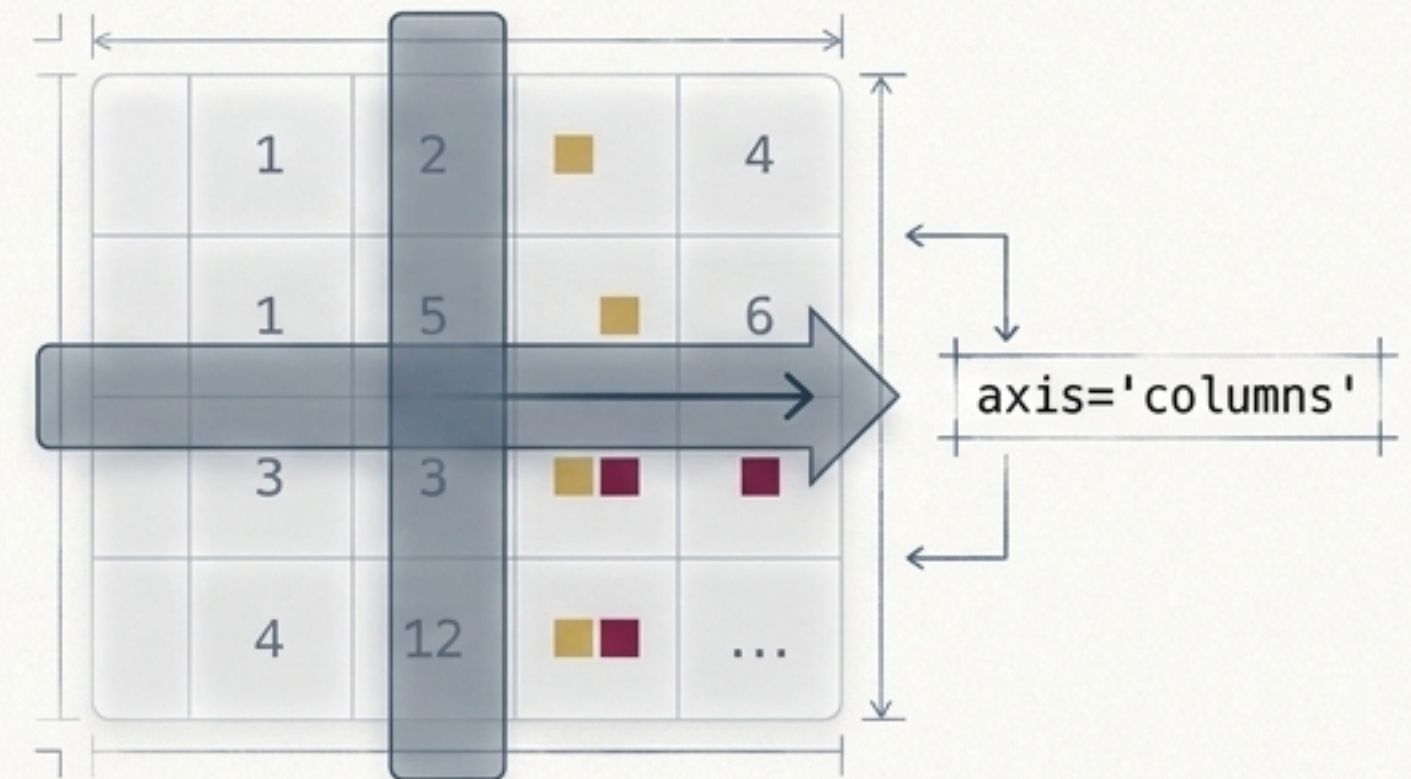
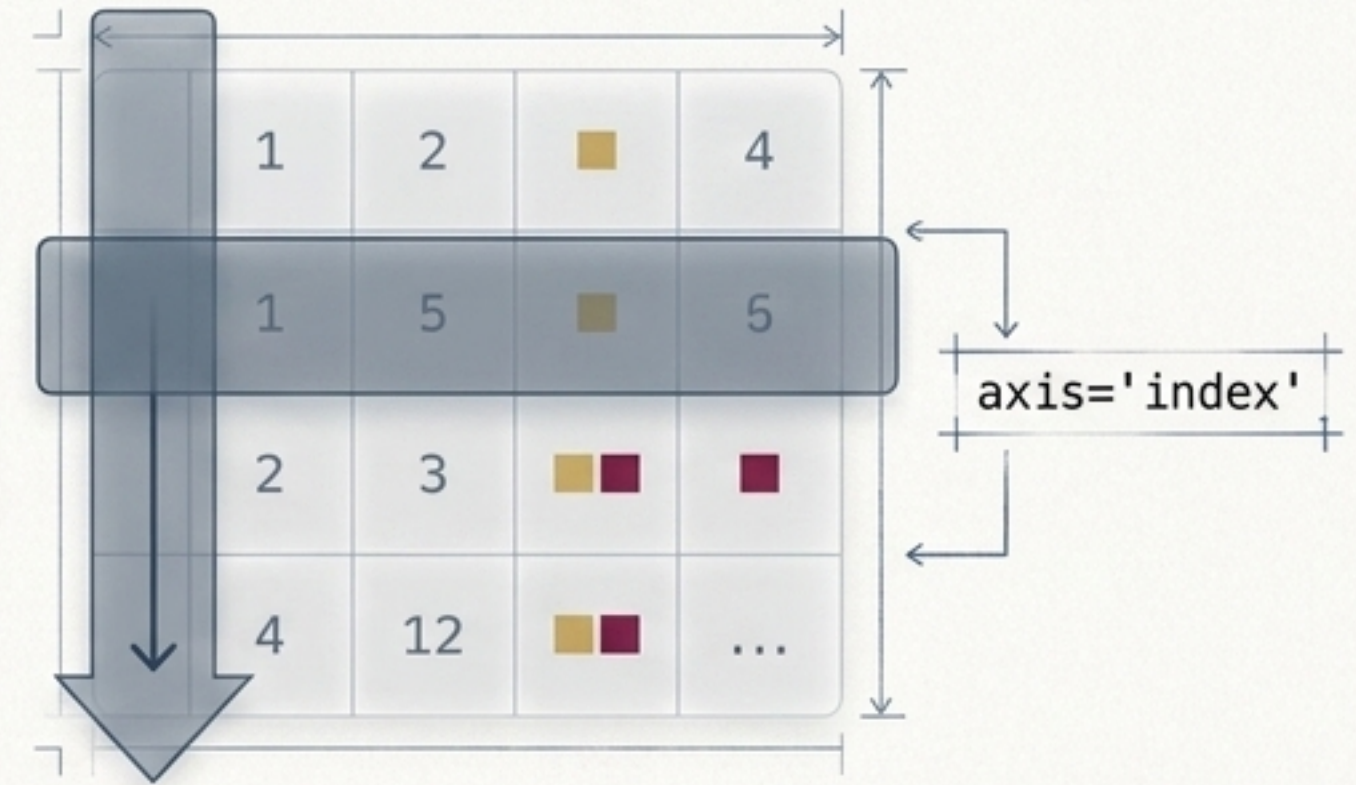
```
review_points_mean = reviews.points.mean()  
reviews.points.map(lambda p: p - review_points_mean)
```



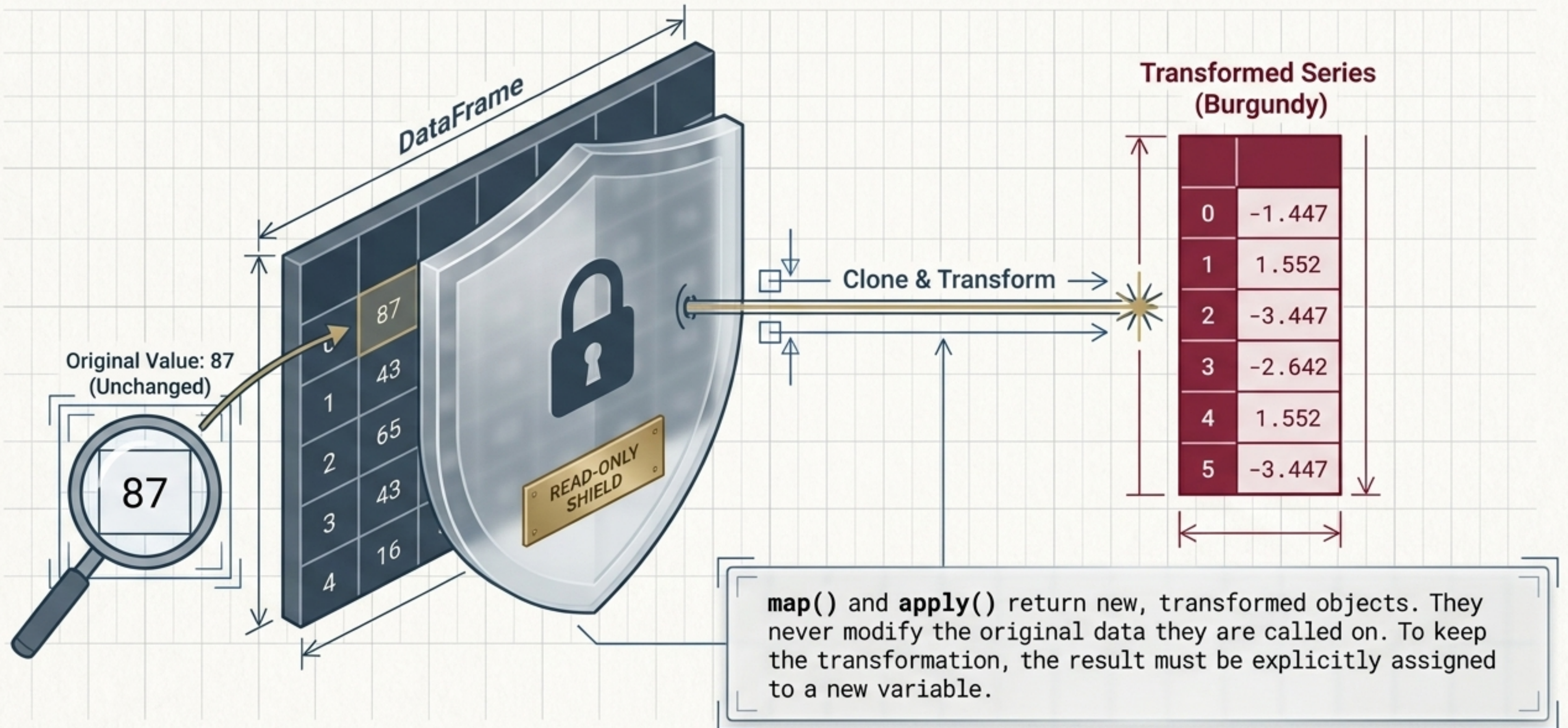
System-wide transformations require apply

`apply()` is used to transform a whole DataFrame by calling a custom method on each row or column, rather than just single values.

```
def remean_points(row):  
    row.points = row.points - review_points_mean  
    return row  
  
reviews.apply(remean_points, axis='columns')
```



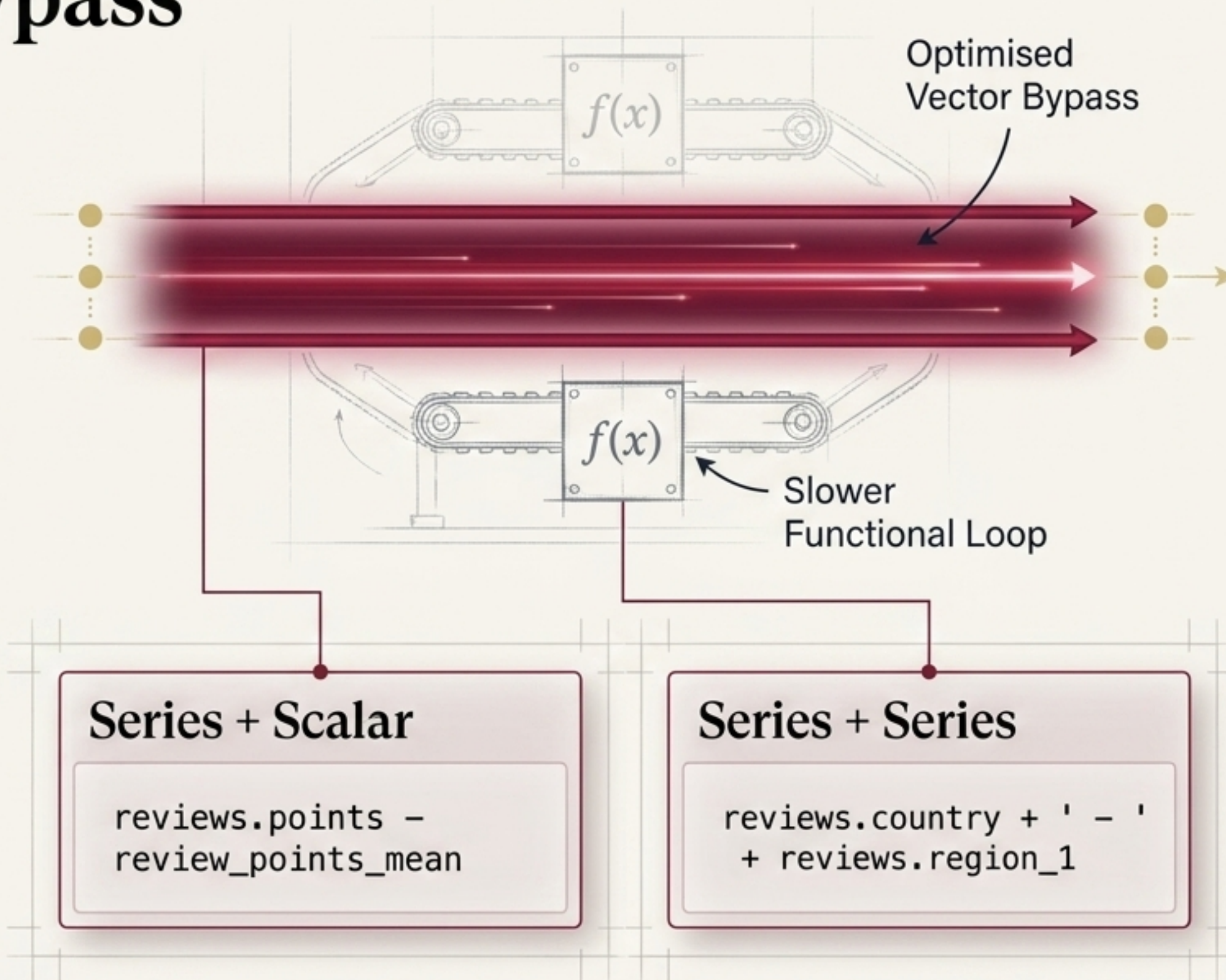
The Golden Rule of Immutability



Built-in operators bypass functional loops

Pandas understands standard Python operators ($>$, $<$, $==$, $+$, $-$).

It **automatically aligns equal-length Series** or **broadcasts scalars** across an **entire Series** using built-in C-level speed-ups.



Selecting the correct transformation tool

	Target	Operation	Flexibility	Speed
map()	Series	Element-wise	High / Custom logic	Standard
apply()	DataFrame	Row / Column-wise	High / Custom logic	Standard
Built-in Operators	Both	Element / Vectorized	Low / Basic math & strings	Highly Optimised

The Engine of Transformation

Summary functions and maps are not isolated tools; they are interlocking gears. Summaries extract the vital parameters of the raw dataset, which maps and operators then use as the specific weights and measures to reshape the data for final analysis.

